Introducción a Transformers

(Enrique Mármol Campos)

Mineria de datos

24 de marzo de 2026

Objetivo de hoy

Entender **cómo funciona un Transformer** construyéndolo pieza a pieza: **embedding** → **atención (Q/K/V)** → **multi-head** → **MLP/FFN** → **bloque Transformer**.

Agenda

1. 1. Tokens y Embeddings (qué son y ejemplo)
2. 2. Atención: de la intuición a Q/K/V (con ejemplos numéricos)
3. 3. Multi-Head Attention (por qué y ejemplo didáctico)
4. 4. MLP / FFN (por qué y ejemplo numérico)
5. 5. El bloque Transformer (montado paso a paso)
6. 6. Salida (muy básico) y cierre

Plan de la sección

1. 1. Tokens y Embeddings (qué son y ejemplo)
2. 2. Atención: de la intuición a Q/K/V (con ejemplos numéricos)
3. 3. Multi-Head Attention (por qué y ejemplo didáctico)
4. 4. MLP / FFN (por qué y ejemplo numérico)
5. 5. El bloque Transformer (montado paso a paso)
6. 6. Salida (muy básico) y cierre

1.1 ¿Qué ve un Transformer? Tokens (no "palabras")

Qué vamos a hacer

Convertir una frase en una secuencia de IDs enteros (tokens).

- Un Transformer opera sobre una secuencia discreta: $w_1, w_2, \ldots, w_N$.
- Esos w_i son **IDs** (enteros), producidos por un tokenizador.

Ejemplo (toy)

`"pollo estaba cansado"` $\rightarrow$ `[pollo, estaba, cansado]` $\rightarrow$ `[1, 2, 3]`

1.2 One-hot: la representación más simple (pero mala)

Qué vamos a hacer

Ver por qué el one-hot no es práctico y por qué usamos embeddings.

- Si el vocabulario tiene $|V| = 6$ tokens, un token se puede representar como vector one-hot de dimensión 6.
- Ejemplo: `pollo` $= [1, 0, 0, 0, 0, 0]$

Problema

$|V|$ real suele ser enorme (50k, 100k, ...): one-hot es **muy grande** y **no captura similitud semántica**.

1.3 Embedding: una tabla aprendible (matriz E)

Qué vamos a hacer

Pasar de IDs a vectores densos con una matriz de embeddings.

- Definimos una matriz $E \in \mathbb{R}^{|V| \times d}$.
- d es la dimensión del embedding (mucho menor que $|V|$).
- Cada token w "selecciona" su fila: $e = E[w]$.

Importante

E se **aprende** durante el entrenamiento (como cualquier otra matriz de pesos).

1.4 Ejemplo numérico: una matriz E de juguete

Qué vamos a hacer

Usar una matriz E (con números) y hacer el lookup del embedding.

Supongamos $|V| = 6$ y $d = 2$. Vocabulario (toy):

`1: pollo 2: estaba 3: cansado 4: carretera 5: era 6: ancha`

Matriz de embeddings E (cada fila es el embedding del token):

$$E = \begin{bmatrix} \texttt{pollo} \\ \texttt{estaba} \\ \texttt{cansado} \\ \texttt{carretera} \\ \texttt{era} \\ \texttt{ancha} \end{bmatrix} = \begin{bmatrix} 1{,}0 & 0{,}0 \\ 0{,}2 & 0{,}1 \\ 0{,}9 & 0{,}0 \\ 0{,}0 & 1{,}0 \\ 0{,}0 & 0{,}0 \\ 0{,}0 & 0{,}9 \end{bmatrix}$$

Lookup

$E[\texttt{pollo}] = [1{,}0,\ 0{,}0], \quad E[\texttt{estaba}] = [0{,}2,\ 0{,}1], \quad E[\texttt{cansado}] = [0{,}9,\ 0{,}0].$

1.5 Ejemplo con frase completa: X como "matriz de la frase"

Qué vamos a hacer

Construir la matriz $X \in \mathbb{R}^{N \times d}$ apilando embeddings de la frase.

Frase: `[pollo, estaba, cansado]` $\Rightarrow N = 3$

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ x_3^\top \end{bmatrix} = \begin{bmatrix} E[\texttt{pollo}] \\ E[\texttt{estaba}] \\ E[\texttt{cansado}] \end{bmatrix} = \begin{bmatrix} 1{,}0 & 0{,}0 \\ 0{,}2 & 0{,}1 \\ 0{,}9 & 0{,}0 \end{bmatrix}$$

Mensaje clave

A partir de aquí, el Transformer ya trabaja con X: una fila por token.

1.6 ¿Y el orden? Positional Embedding/Encoding

Qué vamos a hacer

Añadir información de posición: sin esto, el modelo no sabe el orden.

- Creamos $P \in \mathbb{R}^{N \times d}$ (posiciones 1..N).
- Entrada final: $X_{\text{in}} = X + P$.

Ejemplo (toy) para $N = 3, d = 2$:

$$P = \begin{bmatrix} 0{,}0 & 0{,}1 \\ 0{,}0 & 0{,}2 \\ 0{,}0 & 0{,}3 \end{bmatrix} \quad \Rightarrow \quad X_{\text{in}} = \begin{bmatrix} 1{,}0 & 0{,}1 \\ 0{,}2 & 0{,}3 \\ 0{,}9 & 0{,}3 \end{bmatrix}$$

Nota

En modelos reales: posiciones pueden ser fijas (seno/coseno) o aprendidas.

Plan de la sección

1. 1. Tokens y Embeddings (qué son y ejemplo)
2. 2. Atención: de la intuición a Q/K/V (con ejemplos numéricos)
3. 3. Multi-Head Attention (por qué y ejemplo didáctico)
4. 4. MLP / FFN (por qué y ejemplo numérico)
5. 5. El bloque Transformer (montado paso a paso)
6. 6. Salida (muy básico) y cierre

2.1 ¿Qué hace la atención? (intuición primero)

Qué vamos a hacer

Entender atención como "mezcla ponderada" de tokens.

- Para el token i, creamos una nueva representación a_i.
- a_i es una **suma ponderada** de información de otros tokens:

$$a_i = \sum_{j=1}^{N} \alpha_{i,j} \, v_j$$

- Los pesos $\alpha_{i,j}$ (atención) indican: "¿cuánto mira i a j?"

Dos cosas que necesitamos

(1) Cómo obtener $\alpha_{i,j}$ y (2) qué es exactamente v_j.

2.2 Por qué hace falta contexto (mini ejemplo lingüístico)

Qué vamos a hacer

Ver por qué un embedding "estático" no basta.

- *El pollo no cruzó la carretera porque* ***estaba cansado****.*
- *El pollo no cruzó la carretera porque* ***era demasiado ancha****.*

Idea

Para entender "cansado" quieres mirar a "pollo". Para "ancha" quieres mirar a "carretera".

2.3 Q, K y V: tres roles distintos

Qué vamos a hacer

Definir Q, K, V y su papel.

- Partimos de $X \in \mathbb{R}^{N \times d}$.
- Proyecciones lineales:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- Interpretación:
  - q_i: "qué busco" (Query)
  - k_j: "qué etiqueta tengo" (Key)
  - v_j: "qué información aporto" (Value)

Importante

W_Q, W_K, W_V se **aprenden** durante el entrenamiento.

2.4 Fórmula completa (un head)

Qué vamos a hacer

Ver la atención como un algoritmo en 4 pasos.

Para un token i:

1. **Scores** con producto escalar: $s_{i,j} = q_i \cdot k_j$
2. **Escalado**: $\tilde{s}_{i,j} = \dfrac{s_{i,j}}{\sqrt{d_k}}$
3. **Pesos**: $\alpha_{i,j} = \mathrm{softmax}(\tilde{s}_{i,\cdot})_j$
4. **Mezcla**: $a_i = \sum_j \alpha_{i,j} v_j$

Truco didáctico

En los ejemplos numéricos pondremos $W_Q = W_K = W_V = I$ para poder calcular "a mano".

2.5 ¿Qué es exactamente a_i y para qué sirve?

Idea clave

a_i es el **embedding contextualizado** del token i **después de atención**.

- Antes: x_i era el embedding (token + posición), con poco contexto.
- Atención produce:

$$a_i = \sum_{j=1}^{N} \alpha_{i,j} v_j$$

- Eso significa: el token i ahora "lleva dentro" información de otros tokens relevantes.

No es una predicción

a_i no es probabilidad ni etiqueta. Es una **representación intermedia** que se pasa a las siguientes capas.

2.5 ¿Qué pasa justo después de calcular todos los a_i? (de head a bloque)

Paso 1: la atención se calcula para todos los tokens a la vez

En forma matricial, un head produce:

$$\mathrm{Attn}(X) = \underbrace{\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{A \in \mathbb{R}^{N\times N}} \underbrace{V}_{\in \mathbb{R}^{N\times d_v}} = \underbrace{AV}_{\in \mathbb{R}^{N\times d_v}}$$

Cada fila de AV es un a_i.

Paso 2: si hay multi-head

Cada cabeza produce su AV (sus a_i). Luego:

$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}^{(1)}, \ldots, \mathrm{head}^{(h)})W_O$$

2.5 ¿Qué pasa justo después de calcular todos los a_i? (de head a bloque)

Paso 3: entra en el bloque Transformer

Ese resultado pasa por **residual + LayerNorm** y luego por la **FFN/MLP**. En pre-norm:

$$Y = X + \mathrm{MHA}(\mathrm{LN}(X)), \qquad H = Y + \mathrm{FFN}(\mathrm{LN}(Y))$$

2.5 Ejemplo numérico A: "pollo estaba cansado" (definimos X)

Qué vamos a hacer

Calcular la atención del token **"cansado"** hacia los demás.

Frase: `[pollo, estaba, cansado]` $\Rightarrow N = 3$
Usamos los embeddings toy (d=2):

$$x_{\text{pollo}} = \begin{bmatrix} 1{,}0 \\ 0{,}0 \end{bmatrix}, \; x_{\text{estaba}} = \begin{bmatrix} 0{,}2 \\ 0{,}1 \end{bmatrix}, \; x_{\text{cansado}} = \begin{bmatrix} 0{,}9 \\ 0{,}0 \end{bmatrix}$$

$$X = \begin{bmatrix} 1{,}0 & 0{,}0 \\ 0{,}2 & 0{,}1 \\ 0{,}9 & 0{,}0 \end{bmatrix} \quad \text{y para simplificar:} \quad Q = K = V = X$$

Objetivo

Queremos a_3: la nueva representación del token 3 ("cansado").

2.6 Ejemplo A: Paso 1 (scores) para "cansado"

Qué vamos a hacer

Calcular $s_{3,j} = q_3 \cdot k_j$ con números.

Como $Q = K = X$: $q_3 = x_{\text{cansado}} = [0{,}9, 0]$.

$$s_{3,1} = q_3 \cdot k_1 = [0{,}9, 0] \cdot [1, 0] = 0{,}90$$

$$s_{3,2} = [0{,}9, 0] \cdot [0{,}2, 0{,}1] = 0{,}18$$

$$s_{3,3} = [0{,}9, 0] \cdot [0{,}9, 0] = 0{,}81$$

Interpretación rápida

"cansado" se parece más a "pollo" y a sí mismo que a "estaba".

2.7 Ejemplo A: Paso 2 (escalado) y Paso 3 (softmax)

Qué vamos a hacer

Convertir scores en probabilidades $\alpha_{3,j}$.

Aquí $d_k = 2 \Rightarrow \sqrt{d_k} = \sqrt{2} \approx 1{,}414$.

$$\tilde{s}_{3,1} = 0{,}90/1{,}414 = 0{,}636, \quad \tilde{s}_{3,2} = 0{,}18/1{,}414 = 0{,}127, \quad \tilde{s}_{3,3} = 0{,}81/1{,}414 = 0{,}573$$

Softmax (aprox):

$$(\alpha_{3,1}, \alpha_{3,2}, \alpha_{3,3}) \approx (0{,}394,\ 0{,}237,\ 0{,}370)$$

token j	$\tilde{s}_{3,j}$	$\alpha_{3,j}$	lectura
pollo	0.636	0.394	mira bastante
estaba	0.127	0.237	mira algo
cansado	0.573	0.370	mira también a sí mismo

2.8 Ejemplo A: Paso 4 (mezcla de Values) $\Rightarrow a_3$

Qué vamos a hacer

Construir el nuevo embedding contextual de "cansado".

Como $V = X$, $v_1 = x_{\text{pollo}}, v_2 = x_{\text{estaba}}, v_3 = x_{\text{cansado}}$:

$$a_3 = \sum_{j=1}^{3} \alpha_{3,j} v_j = 0{,}394\, x_{\text{pollo}} + 0{,}237\, x_{\text{estaba}} + 0{,}370\, x_{\text{cansado}}$$

Numéricamente:

$$a_3 \approx 0{,}394 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0{,}237 \begin{bmatrix} 0{,}2 \\ 0{,}1 \end{bmatrix} + 0{,}370 \begin{bmatrix} 0{,}9 \\ 0 \end{bmatrix} = \begin{bmatrix} 0{,}774 \\ 0{,}024 \end{bmatrix}$$

Qué significa

El vector de "cansado" ahora incorpora contexto de "pollo": queda "atado" a quién está cansado.

2.9 Ejemplo A: visual rápido de pesos (token "cansado")

Query: token i = "cansado"

pollo (0.394)

estaba (0.237)

cansado (0.370)

Lectura

El mayor peso cae en **pollo** $\Rightarrow$ "cansado" recupera contexto del sujeto.

2.10 Ejemplo numérico B: "carretera era ancha"

Qué vamos a hacer

Calcular atención para el token **"ancha"** y ver que mira a **"carretera"**.

Frase: `[carretera, era, ancha]` $\Rightarrow N = 3$
Embeddings toy:

$$x_{\text{carretera}} = \begin{bmatrix} 0 \\ 1{,}0 \end{bmatrix}, \; x_{\text{era}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \; x_{\text{ancha}} = \begin{bmatrix} 0 \\ 0{,}9 \end{bmatrix}$$

De nuevo: $Q = K = V = X$. Queremos a_3 para "ancha".

2.11 Ejemplo B: scores $\to$ softmax $\to$ a_3

$q_3 = x_{\text{ancha}} = [0, 0{,}9]$, $d_k = 2 \Rightarrow \sqrt{2}$.
Scores:

$$s_{3,1} = [0, 0{,}9] \cdot [0, 1] = 0{,}90, \quad s_{3,2} = [0, 0{,}9] \cdot [0, 0] = 0{,}00, \quad s_{3,3} = [0, 0{,}9] \cdot [0, 0{,}9] = 0{,}81$$

Escalado:

$$\tilde{s}_{3,\cdot} \approx (0{,}636,\ 0{,}000,\ 0{,}573)$$

Softmax (aprox):

$$(\alpha_{3,1}, \alpha_{3,2}, \alpha_{3,3}) \approx (0{,}405,\ 0{,}214,\ 0{,}380)$$

Salida:

$$a_3 = 0{,}405\, x_{\text{carretera}} + 0{,}214\, x_{\text{era}} + 0{,}380\, x_{\text{ancha}} = \begin{bmatrix} 0 \\ 0{,}748 \end{bmatrix}$$

Lectura

El mayor peso cae en **carretera** $\Rightarrow$ "ancha" se contextualiza con "carretera".

2.12 (Opcional) Máscara causal en el decoder (idea)

Qué vamos a hacer

Ver qué cambia si el modelo no puede mirar al futuro (autoregresivo).

- En generación (decoder), el token i solo puede atender a $j \leq i$.
- Se implementa sumando una máscara M antes del softmax:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)$$

- $M_{i,j} = -\infty$ si $j > i$, y 0 si $j \leq i$.

Plan de la sección

1. 1. Tokens y Embeddings (qué son y ejemplo)
2. 2. Atención: de la intuición a Q/K/V (con ejemplos numéricos)
3. 3. Multi-Head Attention (por qué y ejemplo didáctico)
4. 4. MLP / FFN (por qué y ejemplo numérico)
5. 5. El bloque Transformer (montado paso a paso)
6. 6. Salida (muy básico) y cierre

3.1 ¿Por qué varias cabezas?

Qué vamos a hacer

Entender el problema de "una sola perspectiva" y la solución multi-head.

- Una sola atención produce una sola matriz A (una forma de "mirar").
- Con varias cabezas, el modelo aprende **múltiples formas de atender** en paralelo.
- Intuición: una cabeza puede capturar "sujeto", otra "objeto", otra "concordancia", etc.

3.2 Definición (MHA)

Qué vamos a hacer

Ver la fórmula sin entrar aún en "demasiados detalles".

Para cada cabeza $c = 1, \ldots, h$:

$$Q^{(c)} = XW_Q^{(c)}, \quad K^{(c)} = XW_K^{(c)}, \quad V^{(c)} = XW_V^{(c)}$$

$$\text{head}^{(c)} = \text{softmax}\left(\frac{Q^{(c)}K^{(c)\top}}{\sqrt{d_k}}\right)V^{(c)}$$

Luego:

$$\text{MHA}(X) = \text{Concat}(\text{head}^{(1)}, \ldots, \text{head}^{(h)})W_O$$

Mensaje

Cada cabeza tiene sus W_Q, W_K, W_V aprendibles $\Rightarrow$ cada cabeza aprende "a qué mirar".

3.3 Ejemplo didáctico: dos cabezas miran distinto (misma frase)

Qué vamos a hacer

Ver que dos cabezas pueden producir α diferentes para el mismo token.

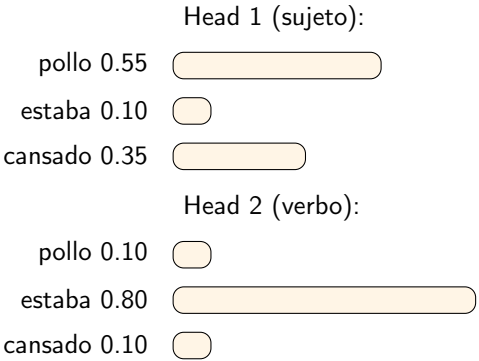
Frase: `[pollo, estaba, cansado]`. Query: token "cansado".

- **Head 1** (relación sujeto): $\alpha^{(1)} \approx [0{,}55,\ 0{,}10,\ 0{,}35]$
- **Head 2** (verbo/auxiliar): $\alpha^{(2)} \approx [0{,}10,\ 0{,}80,\ 0{,}10]$

Interpretación

La misma palabra "cansado" puede mirar a "pollo" (quién) y a "estaba" (estructura verbal) en cabezas distintas.

3.4 Visual rápido: dos cabezas (pesos distintos)



Qué pasa después

Cada cabeza produce su salida $\text{head}^{(c)}$. Se concatenan y se proyectan con W_O.

Plan de la sección

1. 1. Tokens y Embeddings (qué son y ejemplo)
2. 2. Atención: de la intuición a Q/K/V (con ejemplos numéricos)
3. 3. Multi-Head Attention (por qué y ejemplo didáctico)
4. 4. MLP / FFN (por qué y ejemplo numérico)
5. 5. El bloque Transformer (montado paso a paso)
6. 6. Salida (muy básico) y cierre

4.1 ¿Por qué MLP si ya tenemos atención?

Qué vamos a hacer

Entender la división de trabajo: atención mezcla tokens; MLP transforma features.

- La atención **mezcla información entre tokens** (depende de $QK^\top$).
- La MLP/FFN se aplica **por token** (misma red para cada fila).
- Sirve para introducir **no linealidad** y "refinar" representaciones.

4.2 Fórmula de la FFN (posición a posición)

Qué vamos a hacer

Ver la FFN como "expandir y contraer".

$$\mathrm{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

- $W_1 : d \to d_{ff}$ (expande), $W_2 : d_{ff} \to d$ (contrae).
- Típico: $d_{ff} \approx 4d$.
- σ suele ser ReLU o GELU.

4.3 Ejemplo numérico simple (una FFN mini)

Qué vamos a hacer

Aplicar una FFN pequeña a un token con números (para entender la mecánica).

Supón $d = 2$, $d_{ff} = 3$, $\sigma = \text{ReLU}$.

$$x = \begin{bmatrix} 0{,}5 & -1{,}0 \end{bmatrix}$$

$$W_1 = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 & 0{,}5 & 0 \end{bmatrix}$$

Paso 1 (lineal):

$$z = xW_1 + b_1 = \begin{bmatrix} 0{,}5 & -1{,}0 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0{,}5 & 0 \end{bmatrix} = \begin{bmatrix} -1{,}5 & -0{,}5 & -0{,}5 \end{bmatrix}$$

Paso 2 (ReLU):

$$\text{ReLU}(z) = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

Mensaje

La FFN "recombina" features con no linealidad; aquí la salida quedó a cero por el ejemplo.

Plan de la sección

1. 1. Tokens y Embeddings (qué son y ejemplo)
2. 2. Atención: de la intuición a Q/K/V (con ejemplos numéricos)
3. 3. Multi-Head Attention (por qué y ejemplo didáctico)
4. 4. MLP / FFN (por qué y ejemplo numérico)
5. 5. El bloque Transformer (montado paso a paso)
6. 6. Salida (muy básico) y cierre

5.1 Bloque Transformer = (MHA + FFN) + estabilidad

Qué vamos a hacer

Montar el bloque pieza a pieza y entender qué hace cada parte.

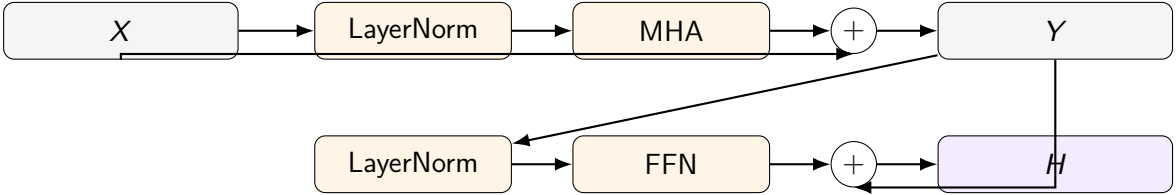
Un bloque típico incluye:

1. **(Multi-head) Self-Attention**
2. **FFN / MLP**
3. **Residual connections** (sumas)
4. **LayerNorm**

Idea clave

Atención → mezcla tokens. FFN → transforma features por token. Residual+Norm → entrenamiento estable.

5.2 Diagrama (pre-norm) de un bloque



Lectura

El residual "salta" por encima del submódulo: $Y = X + \mathrm{MHA}(\mathrm{LN}(X))$ y $H = Y + \mathrm{FFN}(\mathrm{LN}(Y))$.

5.3 ¿Qué parte mezcla tokens?

Qué vamos a aclarar

En un bloque, no todo "mezcla tokens".

- **MHA**: sí mezcla tokens ($QK^\top$ conecta posiciones).
- **LayerNorm**: no mezcla tokens (normaliza cada token por separado).
- **FFN**: no mezcla tokens (se aplica por token).
- **Residual**: solo suma (no mezcla posiciones).

Mensaje clave

La "magia de dependencias largas" está principalmente en la atención.

5.4 Pila de bloques: por qué varias capas

Qué vamos a hacer

Ver por qué apilar capas aumenta capacidad.

- Cada bloque produce $X^{(\ell+1)}$ a partir de $X^{(\ell)}$.
- Con más capas:
  - la representación se vuelve más contextual,
  - aparecen dependencias más abstractas.

$$X^{(0)} \to X^{(1)} \to \cdots \to X^{(L)}$$

Intuición

Capas tempranas: relaciones locales/sintaxis. Capas profundas: semántica/razonamiento (dependiendo de datos/entrenamiento).

Plan de la sección

1. 1. Tokens y Embeddings (qué son y ejemplo)
2. 2. Atención: de la intuición a Q/K/V (con ejemplos numéricos)
3. 3. Multi-Head Attention (por qué y ejemplo didáctico)
4. 4. MLP / FFN (por qué y ejemplo numérico)
5. 5. El bloque Transformer (montado paso a paso)
6. 6. Salida (muy básico) y cierre

6.1 De embeddings a predicción (LM head / clasificación)

Qué vamos a hacer

Entender "qué sale" al final según la tarea.

- **Language Modeling (decoder-only)**: predecir el siguiente token con una proyección a vocabulario:

$$\text{logits} = hW_U + b, \quad p = \text{softmax}(\text{logits})$$

- **Clasificación (encoder-only)**: tomar un token especial (p.ej. [CLS]) y proyectar a clases.

6.X Transformers vs LLMs: ¿qué relación hay?

Transformers

Son una **arquitectura** (una familia de modelos) basada en self-attention + MLP + bloques apilados.

LLMs (Large Language Models)

Son **modelos de lenguaje** (normalmente) basados en Transformers, pero:

- **muy grandes** (muchos parámetros),
- entrenados con **muchísimos datos**,
- y optimizados para tareas de lenguaje (y ahora también multimodales).

Resumen en una frase

Un LLM suele ser un Transformer (casi siempre decoder-only) + entrenamiento a gran escala.

6.X ¿Qué tipo de Transformer suele ser un LLM? (decoder-only)

La mayoría de LLMs modernos: decoder-only Transformer

- Arquitectura: pila de bloques Transformer con **masked self-attention** (causal).
- Objetivo de entrenamiento principal: **next-token prediction**.

Entrenamiento (idea)

Dada una secuencia $w_1, \ldots, w_T$, el modelo aprende:

$$p(w_t \mid w_{<t}) \quad \text{(predecir el siguiente token usando solo el pasado)}$$

¿Y por qué esto da "habilidades"?

Al minimizar ese objetivo sobre datos masivos:

- aprende gramática, hechos, patrones, estilos,
- y representaciones útiles para muchas tareas (zero-shot / few-shot).

Luego se suele hacer **instrucción/alineamiento** (fine-tuning) para que siga instrucciones.

Resumen final (en 8 ideas)

1. Tokens $\rightarrow$ IDs.
2. Embedding = lookup en una matriz E aprendible.
3. Positional embeddings añaden el orden.
4. Atención = suma ponderada de values.
5. Q/K generan pesos (similitud), V aporta el contenido.
6. Softmax convierte scores en probabilidades.
7. Multi-head = varias atenciones en paralelo (perspectivas distintas).
8. Bloque Transformer = MHA + FFN con residual y LayerNorm para estabilidad.